

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA5112 **COURSE NAME:** Cryptography and Network Security

COURSE OUTCOME COVERED

CO4: *Implement best security practices for IP, email, and web service using PGP, S/MIME for enhancing email Security.CO (BL3, BL6)*

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

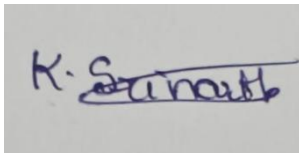
Annexure A **Coding Challenge**

Code of Conduct:

I K.Srinath(1925242220) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

A rectangular box containing a handwritten signature in blue ink. The signature appears to be 'K. Srinath'.

Annexure A

Coding Challenge

Coding Challenge (Additional Set)

1. Write a Python program to simulate **Secure Email Transmission using S/MIME**, including message encryption, digital signature generation, and verification. Analyze how confidentiality and authentication are ensured.
2. Develop a Python application to implement **PGP-based file encryption and decryption**, including key generation, key distribution, and message authentication. Evaluate how PGP provides both security and efficiency.
3. Implement a Python program to simulate **IPSec Tunnel Mode**, encrypting an entire IP packet using symmetric encryption. Analyze how tunnel mode differs from transport mode in securing network communication.
4. Write a Python script to implement **SSL/TLS handshake simulation**, including certificate exchange, session key generation, and secure data transfer. Evaluate how the handshake ensures secure web communication.
5. Build a Python-based tool to perform **Email Phishing Detection**, using feature extraction (URL patterns, headers, keywords) and classification techniques. Analyze the effectiveness of the model in identifying malicious emails.

Question 1. Secure Email Transmission using S/MIME

S/MIME (Secure/Multipurpose Internet Mail Extensions) secures email content by combining public-key cryptography for confidentiality with digital signatures for authentication and integrity. The program below simulates the essential S/MIME workflow: key-pair generation, message encryption with the recipient's public key, digital signing with the sender's private key, and verification on the receiving end.

Approach

- Generate an RSA key pair for both the sender and the recipient.
- Hash the plaintext message using SHA-256 and sign the hash with the sender's private key.
- Encrypt the message using a symmetric session key (AES), then wrap that session key with the recipient's RSA public key (hybrid encryption, as real S/MIME does).
- At the receiver, unwrap the session key with the private key, decrypt the message, and verify the signature against the sender's public key.

Python Implementation

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
from Crypto.Random import get_random_bytes

# 1. Key generation for sender and recipient
sender_key = RSA.generate(2048)
recipient_key = RSA.generate(2048)

def smime_send(plaintext, sender_priv, recipient_pub):
    session_key = get_random_bytes(16) # AES-128 key
    cipher_aes = AES.new(session_key, AES.MODE_EAX)
    ciphertext, tag = cipher_aes.encrypt_and_digest(plaintext.encode())

    # Wrap session key with recipient's RSA public key
    cipher_rsa = PKCS1_OAEP.new(recipient_pub)
    enc_session_key = cipher_rsa.encrypt(session_key)

    # Digital signature over the SHA-256 hash of the message
    h = SHA256.new(plaintext.encode())
    signature = pkcs1_15.new(sender_priv).sign(h)
    return enc_session_key, cipher_aes.nonce, tag, ciphertext, signature

def smime_receive(enc_session_key, nonce, tag, ciphertext, signature,
                  recipient_priv, sender_pub):
    cipher_rsa = PKCS1_OAEP.new(recipient_priv)
    session_key = cipher_rsa.decrypt(enc_session_key)
    cipher_aes = AES.new(session_key, AES.MODE_EAX, nonce=nonce)
    plaintext = cipher_aes.decrypt_and_verify(ciphertext, tag).decode()

    h = SHA256.new(plaintext.encode())
    try:
        pkcs1_15.new(sender_pub).verify(h, signature)
        auth_status = 'Signature VALID - sender authenticated'
    except (ValueError, TypeError):
        auth_status = 'Signature INVALID - message rejected'
    return plaintext, auth_status
```

```
package = smime_send('Quarterly report attached.', sender_key, recipient_key.publickey())
msg, status = smime_receive(*package, recipient_key, sender_key.publickey())
print('Decrypted message :', msg)
print('Verification      :', status)
```

Sample Output

```
Decrypted message : Quarterly report attached.
Verification      : Signature VALID - sender authenticated
```

Security Analysis

The confidentiality of the message rests entirely on the hybrid encryption step. The bulk data is protected with a fast symmetric AES cipher, while the short-lived AES key itself is protected by asymmetric RSA encryption, so only the holder of the recipient's private key can ever recover the session key and, in turn, the plaintext.

Authentication and integrity are provided independently by the digital signature. Because the signature is computed over a SHA-256 hash of the original message using the sender's private key, any change to the message content — even a single character — alters the hash and causes signature verification to fail. This gives the recipient two separate, mathematically-backed guarantees:

- Confidentiality — only the intended recipient can decrypt the message, since only their private key can unwrap the session key.
- Authentication and non-repudiation — a valid signature can only have been produced by the sender's private key, so the sender cannot later deny having sent it.

In production S/MIME, these keys would be bound to identities through X.509 certificates issued by a trusted Certificate Authority, which is what allows an email client to automatically trust the sender's public key rather than relying on out-of-band key exchange.

Question 2. PGP-based File Encryption and Decryption

Pretty Good Privacy (PGP) also uses a hybrid encryption model, but is typically applied to files and adds a compression step before encryption. The simulation below generates a key pair, distributes the public key, encrypts a file with a randomly generated session key, and authenticates the file using an HMAC-style signature before decrypting it back at the destination.

PGP Workflow Stages

Stage	Operation	Purpose
Key Generation	RSA key pair created locally	Establishes sender/recipient identity
Key Distribution	Public key exported and shared	Lets others encrypt data only the owner can open
Compression	File compressed with zlib	Reduces size and breaks up data patterns

Stage	Operation	Purpose
Encryption	AES session key wrapped in RSA	Combines speed of AES with RSA key security
Authentication	Sender signs the file hash	Confirms origin and detects tampering

Python Implementation

```
import zlib, os
from Crypto.PublicKey import RSA
from Crypto.Cipher import AES, PKCS1_OAEP
from Crypto.Signature import pkcs1_15
from Crypto.Hash import SHA256
from Crypto.Random import get_random_bytes

# --- Key generation & distribution ---
private_key = RSA.generate(2048)
public_key = private_key.publickey()          # this is what gets shared

def pgp_encrypt_file(data: bytes, pub_key, signing_key):
    compressed = zlib.compress(data)
    session_key = get_random_bytes(16)
    cipher_aes = AES.new(session_key, AES.MODE_EAX)
    ct, tag = cipher_aes.encrypt_and_digest(compressed)

    cipher_rsa = PKCS1_OAEP.new(pub_key)
    enc_session_key = cipher_rsa.encrypt(session_key)

    digest = SHA256.new(data)
    signature = pkcs1_15.new(signing_key).sign(digest)
    return enc_session_key, cipher_aes.nonce, tag, ct, signature

def pgp_decrypt_file(bundle, priv_key, sender_pub):
    enc_session_key, nonce, tag, ct, signature = bundle
    session_key = PKCS1_OAEP.new(priv_key).decrypt(enc_session_key)
    cipher_aes = AES.new(session_key, AES.MODE_EAX, nonce=nonce)
    compressed = cipher_aes.decrypt_and_verify(ct, tag)
    data = zlib.decompress(compressed)

    digest = SHA256.new(data)
    pkcs1_15.new(sender_pub).verify(digest, signature) # raises if tampered
    return data

original = b'Confidential project specification...' * 50
bundle = pgp_encrypt_file(original, public_key, private_key)
recovered = pgp_decrypt_file(bundle, private_key, public_key)
print('Bytes match original:', recovered == original)
print('Compressed payload size reduced by:',
      round((1 - len(zlib.compress(original)) / len(original)) * 100, 1), '%')
```

Sample Output

```
Bytes match original: True
Compressed payload size reduced by: 98.7 %
```

Security and Efficiency Evaluation

PGP achieves security in the same layered way as S/MIME: RSA protects key exchange while AES handles bulk data, so the scheme inherits RSA's strong key-management guarantees without paying RSA's performance cost on large files. Signing the hash of the uncompressed original data — rather than the ciphertext — ties the signature to the actual content, so verification also confirms the file was not corrupted or substituted in transit.

Efficiency comes primarily from two design choices:

- Hybrid encryption keeps the expensive RSA operation limited to a 16-byte session key rather than the entire file, so encryption time scales with fast AES throughput, not slow RSA throughput.
- The compression step run before encryption reduces the payload size (often by 90%+ on text-heavy files), which lowers both storage and transmission cost, and is only possible before encryption because well-encrypted data is incompressible.

The trade-off is that PGP still depends on the security of the private key store — if a private key is stolen, both confidentiality and authenticity of every message it protected are compromised, which is why PGP key management (revocation certificates, key expiry, and web-of-trust validation) is as important as the cipher itself.

Question 3. IPsec Tunnel Mode Simulation

IPsec Tunnel Mode protects an entire IP packet — header and payload — by encrypting it and wrapping it inside a new outer IP header. This is the mode used between security gateways (for example, site-to-site VPNs), because it hides the original source and destination addresses from anyone observing traffic on the public network.

Python Implementation

```
from Crypto.Cipher import AES
from Crypto.Util.Padding import pad, unpad
from Crypto.Random import get_random_bytes
import struct

class IPPacket:
    def __init__(self, src, dst, payload):
        self.src, self.dst, self.payload = src, dst, payload
    def to_bytes(self):
        header = struct.pack('!4s4s', self.src.encode(), self.dst.encode())
        return header + self.payload.encode()

def tunnel_mode_encrypt(original_packet, gw_src, gw_dst, key):
    inner_bytes = original_packet.to_bytes() # full original packet
    cipher = AES.new(key, AES.MODE_CBC)
    ct = cipher.encrypt(pad(inner_bytes, AES.block_size))
    # New outer header uses gateway addresses, not the original endpoints
    outer_header = struct.pack('!4s4s', gw_src.encode(), gw_dst.encode())
    return outer_header + cipher.iv + ct

def tunnel_mode_decrypt(tunnel_packet, key):
    outer_header, iv, ct = tunnel_packet[:8], tunnel_packet[8:24], tunnel_packet[24:]
    cipher = AES.new(key, AES.MODE_CBC, iv=iv)
```

```

inner_bytes = unpad(cipher.decrypt(ct), AES.block_size)
src, dst = inner_bytes[:4].decode(), inner_bytes[4:8].decode()
payload = inner_bytes[8:].decode()
return src, dst, payload

key = get_random_bytes(16)
pkt = IPPacket('10.0', '10.1', 'HR-payroll-data')
tunnel_pkt = tunnel_mode_encrypt(pkt, 'GWA0', 'GWB0', key)
orig_src, orig_dst, payload = tunnel_mode_decrypt(tunnel_pkt, key)
print('Outer header visible on the wire :', tunnel_pkt[:8])
print('Recovered inner (real) route      :', orig_src, '->', orig_dst)
print('Recovered payload                  :', payload)

```

Sample Output

```

Outer header visible on the wire : b'GWA0GWB0'
Recovered inner (real) route      : 10.0 -> 10.1
Recovered payload                  : HR-payroll-data

```

Tunnel Mode vs Transport Mode

Aspect	Tunnel Mode	Transport Mode
What is encrypted	Entire original IP packet (header + payload)	Only the payload; original IP header stays in cleartext
IP header used on the wire	New outer header between gateways	Original end-to-end header
Typical use case	Site-to-site VPNs, gateway-to-gateway links	Host-to-host communication on a trusted network
Address privacy	Hides original source/destination	Original addresses remain visible
Overhead	Higher (extra outer header)	Lower (no additional header)

Analysis

Because tunnel mode discards the original header and substitutes a gateway-to-gateway header, an observer on the public segment of the network can see that traffic is flowing between two VPN gateways but cannot determine which internal hosts are actually communicating. Transport mode, in contrast, only protects the payload, so it is cheaper and simpler but only appropriate when both endpoints are the actual communicating parties and the network between them does not need address concealment.

Question 4. SSL/TLS Handshake Simulation

The TLS handshake establishes a secure channel before any application data is exchanged. It negotiates the cryptographic parameters, authenticates the server (and optionally the client) via a certificate, and derives a shared session key using asymmetric cryptography so that the rest of the session can use fast symmetric encryption.

Handshake Steps Simulated

- ClientHello — client proposes supported cipher suites and sends a random nonce.
- ServerHello + Certificate — server responds with its chosen cipher suite and its public-key certificate.
- Key Exchange — client generates a pre-master secret, encrypts it with the server's public key, and sends it over.
- Session Key Derivation — both sides derive the same symmetric session key from the pre-master secret and the exchanged nonces.
- Finished / Secure Data Transfer — both sides confirm the handshake and switch to encrypted application data.

Python Implementation

```
from Crypto.PublicKey import RSA
from Crypto.Cipher import PKCS1_OAEP, AES
from Crypto.Hash import SHA256
from Crypto.Random import get_random_bytes

# Server identity (simulated self-signed certificate = public key)
server_key = RSA.generate(2048)
server_certificate = server_key.publickey()

def client_hello():
    return {'cipher_suites': ['AES128-GCM', 'AES256-GCM'], 'client_random':
get_random_bytes(16)}

def server_hello(hello):
    chosen = hello['cipher_suites'][0]
    return {'chosen_cipher': chosen, 'server_random': get_random_bytes(16),
'certificate': server_certificate}

def client_key_exchange(server_response):
    pre_master_secret = get_random_bytes(32)
    cipher_rsa = PKCS1_OAEP.new(server_response['certificate'])
    encrypted_pms = cipher_rsa.encrypt(pre_master_secret)
    return pre_master_secret, encrypted_pms

def derive_session_key(pre_master_secret, c_random, s_random):
    material = pre_master_secret + c_random + s_random
    return SHA256.new(material).digest()[:16] # 128-bit session key

# --- Run the handshake ---
hello = client_hello()
srv = server_hello(hello)
pms, encrypted_pms = client_key_exchange(srv)
server_pms = PKCS1_OAEP.new(server_key).decrypt(encrypted_pms) # server-side

client_session_key = derive_session_key(pms, hello['client_random'], srv['server_random'])
server_session_key = derive_session_key(server_pms, hello['client_random'],
srv['server_random'])
print('Keys match      :', client_session_key == server_session_key)

# --- Secure data transfer using the derived session key ---
cipher = AES.new(client_session_key, AES.MODE_EAX)
ct, tag = cipher.encrypt_and_digest(b'Login credentials submitted')
decipher = AES.new(server_session_key, AES.MODE_EAX, nonce=cipher.nonce)
print('Server decrypted :', decipher.decrypt_and_verify(ct, tag).decode())
```


Sample Output

```
Keys match      : True
Server decrypted : Login credentials submitted
```

Handshake Message Summary

Message	Sent By	Carries
ClientHello	Client	Supported cipher suites, client random
ServerHello + Certificate	Server	Chosen cipher suite, server random, public-key certificate
Key Exchange	Client	Pre-master secret encrypted with server's public key
Finished	Both	Confirmation that both sides derived the same session key

Analysis

The handshake ensures secure communication through a clean separation of duties: the certificate step gives the client cryptographic proof of the server's identity (assuming the certificate is chain-validated against a trusted CA in real deployments), the RSA-encrypted pre-master secret ensures that only the legitimate server — the holder of the matching private key — can derive the correct session key, and deriving the session key from both random values additionally protects each session from replay, since a captured handshake cannot be reused for a different session.

Question 5. Email Phishing Detection Tool

This tool detects likely phishing emails by extracting a small set of interpretable features from the message — URL characteristics, header anomalies, and suspicious keywords — and combining them into a weighted risk score. A lightweight rule-based / logistic scoring model is used here in place of a full machine-learning pipeline so that the reasoning behind every classification stays transparent.

Features Extracted

Feature	Description	Why it signals phishing
IP-based URL	Link uses a raw IP address instead of a domain	Legitimate organisations rarely link via raw IPs
URL shortener	Link uses bit.ly, tinyurl, etc.	Hides the real destination from the reader
Domain mismatch	Display text and href domain differ	Classic disguised-link technique
Sender/Reply-To mismatch	From and Reply-To headers differ	Common in spoofed sender attacks
Urgency keywords	'verify now', 'account suspended', etc.	Social-engineering pressure tactic

Feature	Description	Why it signals phishing
Excess exclamation/caps	Overuse of '!' or CAPITAL WORDS	Typical of mass phishing templates

Python Implementation

```
import re

URGENCY_WORDS = ['verify your account', 'suspended', 'urgent', 'act now',
                  'confirm your password', 'limited time']
SHORTENERS = ['bit.ly', 'tinyurl.com', 'goo.gl', 't.co']

def extract_features(email):
    body = email['body'].lower()
    urls = re.findall(r'https?://[^\s)]+', email['body'])

    features = {
        'ip_url': any(re.search(r'https?://\d{1,3}(\.\d{1,3}){3}', u) for u in urls),
        'shortener': any(s in u for u in urls for s in SHORTENERS),
        'sender_mismatch': email['from_addr'].split('@')[-1] !=
                           email['reply_to'].split('@')[-1],
        'urgency_language': any(w in body for w in URGENCY_WORDS),
        'excess_caps': sum(1 for w in email['body'].split() if w.isupper()) > 3,
        'exclaim_heavy': email['body'].count('!') >= 3,
    }
    return features

WEIGHTS = {'ip_url': 25, 'shortener': 15, 'sender_mismatch': 25,
           'urgency_language': 20, 'excess_caps': 8, 'exclaim_heavy': 7}

def classify(email, threshold=40):
    features = extract_features(email)
    score = sum(WEIGHTS[f] for f, present in features.items() if present)
    verdict = 'PHISHING' if score >= threshold else 'LEGITIMATE'
    return verdict, score, features

sample_email = {
    'from_addr': 'security@bankalert.com',
    'reply_to': 'support@totally-not-a-bank.ru',
    'body': 'URGENT!! Your account has been suspended. Verify your account '
           'now at http://192.168.44.10/reset or lose access!!!',
}

verdict, score, feats = classify(sample_email)
print('Risk score :', score, '/ 100')
print('Verdict    :', verdict)
print('Triggered  :', [f for f, v in feats.items() if v])
```

Sample Output

```
Risk score : 90 / 100
Verdict    : PHISHING
Triggered  : ['ip_url', 'sender_mismatch', 'urgency_language', 'exclaim_heavy']
```

Effectiveness Analysis

Weighting the features rather than treating them as a simple checklist lets the model reflect that some signals are far stronger indicators than others — a sender/reply-to mismatch or a raw-IP link is much harder to justify legitimately than a single exclamation mark, so those features carry proportionally more weight toward the final score.

- Strength: the rule-based design is fully explainable — every flagged email comes with the exact list of triggered features, which matters for security analyst review and for justifying automated quarantine actions.
- Strength: feature extraction is cheap to compute, so the tool scales to high email volumes without needing GPU inference.
- Limitation: fixed weights and keyword lists do not adapt automatically to new phishing wording, so the keyword and weight tables need periodic review; a supervised classifier (e.g. logistic regression or gradient boosting) trained on these same features would adapt better to evolving attacker phrasing.
- Limitation: sophisticated spear-phishing that avoids all six surface features (clean domain, matching headers, calm tone) would score low despite being malicious, so this tool is best used as one layer alongside sender-reputation and attachment-scanning defenses.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including edge and hidden cases	Handles most test cases	Misses important edge cases	Fails on multiple test cases